



Security Assessment



Aave a.DI

June 2026

Prepared for Aave Labs

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	4
Assessment Methodology.....	6
Threat and Security Overview.....	7
Findings Summary.....	9
Severity Matrix.....	9
Detailed Findings.....	10
Low Severity Issues.....	11
L-01: Fees get left in CrossChainController for each failed bridge send.....	11
L-02: retryEnvelope does not enforce any minimum forwarding-success count.....	13
L-03: Misconfiguration can cause IOUs to be burned while no message is delivered to destination chain	14
Informational Severity Issues.....	16
I-01: requiredForwardingSuccessesByChain can exceed the live bridge count bricking forwarding.....	16
I-02: Unused imports, redundant cast, and stale NatSpec across bridge adapters.....	18
I-03: Cross-block quote/forward fee mismatch can revert sends or strand funds, since random bridge selection changes each block.....	20
Disclaimer.....	21
About Certora.....	21

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
a.DI	https://github.com/aave/aave-delivery-infrastructure	2a66a15 (initial) a044d20 (fix)	Solidity

Project Overview

This document describes the security review of **a.DI**. The work was undertaken from **May 25th, 2026** to **June 1st, 2026**.

The following contract list is included in our scope:

```
src/contracts/CrossChainForwarder.sol
src/contracts/CrossChainReceiver.sol
src/contracts/adapters/IBaseAdapter.sol
src/contracts/adapters/IConfigurableAdapter.sol
src/contracts/adapters/arbitrum/ArbAdapter.sol
src/contracts/adapters/ccip/CCIPAdapter.sol
src/contracts/adapters/ccip/lib/CCIPChainSelectors.sol
src/contracts/adapters/ccip/lib/Client.sol
src/contracts/adapters/gnosisChain/GnosisChainAdapter.sol
src/contracts/adapters/hyperLane/HyperLaneAdapter.sol
src/contracts/adapters/layerZero/ILayerZeroAdapter.sol
src/contracts/adapters/layerZero/LayerZeroAdapter.sol
src/contracts/adapters/layerZero/interfaces/ILayerZeroEndpointV2.sol
src/contracts/adapters/linea/LineaAdapter.sol
src/contracts/adapters/metis/MetisAdapter.sol
src/contracts/adapters/optimism/OpAdapter.sol
src/contracts/adapters/polygon/PolygonAdapterBase.sol
src/contracts/adapters/sameChain/SameChainAdapter.sol
```

```
src/contracts/adapters/scroll/ScrollAdapter.sol
src/contracts/adapters/wormhole/WormholeAdapter.sol
src/contracts/adapters/zkEVM/ZkEVMAdapter.sol
src/contracts/adapters/zkSync/ZkSyncAdapter.sol
src/contracts/interfaces/IBaseReceiverPortal.sol
src/contracts/interfaces/ICrossChainForwarder.sol
src/contracts/libs/Errors.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered edge cases in the code, as listed on the following page.

Protocol Overview

Aave Delivery Infrastructure (or a.DI), is a cross-chain messaging layer used to route governance and protocol messages between networks Aave is deployed on. The system integrates multiple bridge providers by encoding each outbound message into a canonical envelope and transaction format, forwarding it through configured bridge adapters, and requiring receiver-side confirmation rules before a message is delivered to its final destination.

The main entry point is the [CrossChainController.sol](#), which combines message forwarding, message receiving, adapter configuration, retry logic, fee funding, and administrative controls. On the destination chain, the receiver only accepts messages from approved bridge adapters for the relevant origin chain, tracks which adapters have delivered a given transaction, and delivers the enclosed envelope only once the configured confirmation threshold is reached.

This audit was focused on the changes introduced in the tag [audit-snapshot-2026-05-18](#), represented by the following [diff](#).

The audited changes primarily introduced changes to the existing [CrossChainForwarder.sol](#). The forwarder tracks approved message senders, envelope and transaction nonces, registered envelopes, forwarded transaction IDs, per-chain bridge adapter configurations, optimal forwarding bandwidth, and optional source-side forwarding success requirements. Approved senders call [forwardMessage\(\)](#), which encodes the message into an envelope, wraps it in a transaction, registers both identifiers, selects the applicable adapters for the destination chain, and attempts to forward the transaction through each adapter via `delegatecall`.

The forwarder supports configurable bridge selection through per-chain [optimalBandwidth](#). If no limit is set, all configured adapters for the destination chain are used; otherwise, the forwarder

selects a bounded subset of adapters from the available set. The new `requiredForwardingSuccesses` setting adds a source-chain check requiring a minimum number of adapter forwarding calls to succeed before the outbound message is considered accepted, while receiver-side confirmations remain the authoritative delivery quorum.

The new forwarder also introduces retry flows. `retryEnvelope()` re-sends a previously registered envelope as a new transaction, which is useful after receiver-side invalidation or recovery events. `retryTransaction()` re-sends an already forwarded transaction through selected adapters, allowing operators to recover from partial forwarding failures, bridge outages, or insufficient gas/funding on the initial attempt.

Supporting changes were made to adapter interfaces and implementations to expose quote functions. The forwarder's quote methods simulate the same encoded payloads used for forwarding or retries, query the selected adapters, aggregate native and ERC20-denominated fees, and report how many adapter quotes succeeded. This allows callers and operators to estimate bridge costs before sending while preserving the existing adapter model, where outbound bridge calls execute in the controller context and inbound messages are verified through adapter-specific trusted remote checks.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The security model centers on a.DI as a privileged cross-chain message transport layer. The key trust boundaries are between approved message senders, the source-chain [CrossChainController/CrossChainForwarder](#), bridge adapter implementations and external bridge providers, the destination-chain [CrossChainReceiver](#), and final receiver contracts. The main assumptions are that owner/guardian roles are correctly controlled, configured adapters match trusted remote contracts, bridge-specific verification is sound, and receiver-side confirmation thresholds are sufficient for the risk of the enabled bridge set.

This was a manual diff review. The audit team read the relevant Solidity code and reviewed it for logic errors, incorrect assumptions, access-control bypasses, unsafe external calls, configuration hazards, fee/payment issues, replay paths, and inconsistencies between the forwarder, adapters, and receiver. Specific items checked included:

- approved sender, owner, and guardian permissions
- adapter enable/disable and destination adapter configuration
- envelope and transaction ID uniqueness, registration, and retry behavior
- source-side forwarding success requirements
- receiver-side confirmation and duplicate-delivery protections
- quote functions and fee aggregation across native/ERC20 fees
- adapter delegatecall behavior and bridge-specific quote/send consistency
- emergency and recovery paths affecting forwarder configuration

The primary properties reviewed were that only approved senders can originate new messages, only configured adapters are used for a destination chain, retries are limited to previously registered envelopes or forwarded transactions, receiver delivery occurs at most once per envelope, and adapter failures do not silently break configured safety assumptions. The review also considered whether quote functions could mislead callers, whether failed adapter calls could leave inconsistent forwarding state, whether bandwidth settings could reduce security

below intended thresholds, and whether external bridge dependencies or fee-token flows could create DoS or value drain risks.

The medium issue found was a case where misconfiguration can cause IOUs to be burned while no message is delivered to the destination chain.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	3	3	1
Informational	3	3	–
Total	6	6	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Fees get left in CrossChainController for each failed bridge send	Low	Acknowledged
L-02	retryEnvelope does not enforce any minimum forwarding-success count	Low	Acknowledged
L-03	Misconfiguration can cause IOUs to be burned while no message is delivered to destination chain	Low	Fixed
I-01	requiredForwardingSuccessesByChain can exceed the live bridge count bricking forwarding	Informational	Acknowledged
I-02	Unused imports, redundant cast, and stale NatSpec across bridge adapters	Informational	Acknowledged
I-03	Cross-block quote/forward fee mismatch can revert sends or strand funds, since random bridge selection changes each block	Informational	Acknowledged

Low Severity Issues

L-01: Fees get left in CrossChainController for each failed bridge send

Severity: Low	Impact: Low	Likelihood: Medium
Files: aave-delivery-infrastructure/ src/contracts/CrossChainFor warder.sol	Status: Acknowledged	

Description:

When IOUs are bridged through multiple aDI adapters, `AdiAdapter._fundCrossChainController` pulls the aggregated fees for *all* selected adapters into the CrossChainController up front. If the CCIP leg's `ccipSend` reverts but other legs satisfy `requiredForwardingSuccessesByChain`, `forwardMessage` returns successfully.

The bridge fee was already moved into the CCC but never spent, and the `AdiAdapter` only refunds native (`_refundExcessNative`) – there is no bridge fee token refund path. The fee is stranded in the CCC. The bridged IOU is still delivered via the surviving bridge, so the loss is bound to the wasted fee.

Recommendations:

After `forwardMessage`, refund any unspent ERC-20 fee to the `feePayer` (compare CCC fee-token balance before/after, or reconcile against actual forwarding successes), or document that stranded fees accumulate in the CCC and following calls are free to spend them.

Customer Response:

Acknowledged, won't fix (documentation only). The impact is bounded to the wasted fee of the failed leg and there is no loss of the IOU itself, no impact on the broader system. The required-successes threshold guarantees delivery is unaffected. We consider the worst case (a stranded, dust-level fee accruing in the CCC) to be an acceptable cost relative to the added complexity and surface area of a per-leg fee reconciliation/refund mechanism, which would

require comparing CCC fee-token balances before/after or reconciling against actual forwarding successes on every `forwardMessage`.

L-02: retryEnvelope does not enforce any minimum forwarding-success count

Severity: Low	Impact: Low	Likelihood: Low
Files: aave-delivery-infrastructure/ src/contracts/CrossChainForwarder.sol	Status: Acknowledged	

Description:

The three send paths in `CrossChainForwarder` apply inconsistent success thresholds after fanning the transaction out to the bridge adapters in `_bridgeTransaction`:

- `forwardMessage` requires `forwardingSuccesses >= _requiredForwardingSuccessesByChain[destinationChainId]`, reverting the whole send (and rolling back envelope/transaction registration) if too few bridges accept.
- `retryTransaction` requires `forwardingSuccesses > 0`.
- `retryEnvelope` has **no** success check at all. It calls `_bridgeTransaction`, registers the new `transactionId`, and returns it even if **zero** bridge adapters succeeded.

As a result, a caller can invoke `retryEnvelope`, pay the bridge fees while no bridge operation executes successfully.

Recommendations:

Consider making `retryEnvelope` consistent with `retryTransaction` by requiring `forwardingSuccesses > 0`.

Customer Response:

Acknowledged; this is by design. A retry may need to send through only one missing or additional adapter, so no minimum success count is required. A retry also does not assume any change in source state, so no success check is necessary.

L-03: Misconfiguration can cause IOUs to be burned while no message is delivered to destination chain

Severity: Low	Impact: Medium	Likelihood: Low
Files: aave-delivery-infrastructure/ src/contracts/CrossChainForwarder.sol	Status: Fixed	

Description:

`_requiredForwardingSuccessesByChain` defaults to `0` and is never initialized by the constructor or by `enableBridgeAdapters`. Raising it requires a separate `updateRequiredForwardingSuccessesByChain` call.

Every time an adapter is added or a new destination chain is configured, ops must remember an extra step to set it to ≥ 1 . If they forget, the chain stays at `0` – a value the setter explicitly allows (no lower bound; documented as "disables strict forwarding").

When it is `0`, `forwardMessage` passes `require(forwardingSuccesses >= 0)` even when **every** bridge adapter fails, and returns normally.

The stable-vault `AdiAdapter` treats that return as proof of dispatch, so `louTokenManager.bridgeTokens` finalizes: the user's IOU is burned/locked and the fee is pulled, **but no cross-chain message is delivered**.

The IOU is lost with nothing on the destination chain, and the `AdiAdapter` cannot detect this since `forwardMessage`'s return value carries no success count.

Recommendations:

1. Forbid `0` for any configured chain: in `_enableBridgeAdapters`, do not allow a chain to have ≥ 1 adapter while `_requiredForwardingSuccessesByChain == 0`, and reject `0` in `_updateRequiredForwardingSuccessesByChain (require(requiredSuccesses >= 1))`.

2. Enforce it again in `forwardMessage:`
`require(_requiredForwardingSuccessesByChain[destinationChainId] >= 1)` so a forward can never succeed with zero successful bridges, regardless of config ordering.

Customer Response:

The original a.DI defaults to 0 successes, which is the behaviour prior to this change, and that default is intentionally preserved. The issue was instead addressed on the Stable Vaults side (the `AdiAdapter` and deployment scripts) in [PR#349](#).

Fix Review:

Fixed in [PR#349](#).

Informational Severity Issues

I-01: requiredForwardingSuccessesByChain can exceed the live bridge count bricking forwarding

Files:

aave-delivery-infrastructure/src/contracts/CrossChainForwarder.sol

Description:

`_requiredForwardingSuccessesByChain[chainId]` is validated `<= effectiveAdapterCount` only at the moment `updateRequiredForwardingSuccessesByChain` is called.

Nothing re-validates it afterward: `disableBridgeAdapters` and `_updateOptimalBandwidthByChain` change the effective bridge count without adjusting it. So a config sequence of setting `required = 2` with 2 bridges, later disable one – leaves `required (2) > effective count (1)`.

From then on `forwardMessage` to that chain can never reach the threshold and reverts every time, bricking all IOU bridging to that destination until governance fixes the value. No funds are lost (the source tx reverts atomically), but bridging is unavailable.

Recommendations:

Disabling a bridge adapter is the only way of reducing the required forwarding successes, since changing the required forwarding successes prevents a state where the required number of adapters is greater than the existing amount.

So preventing disabling when the resulting amount is less than the required amount would create a deadlock where the admin can never decrease the required number of adapters.

However, this issue might silently disable bridging, if the admin doesn't realize that disabling a specific adapter will make the resulting number of adapters less than the required amount.

One way of handling this would be to add a boolean param `checkRequiredSuccesses` to the `disableBridgeAdapters` function that:

- If `true`, revert if the resulting number of adapters is less than the required amount
- If `false`, keep the behavior as is



Customer Response:

Acknowledged, won't fix. Mitigation responsibility falls to off-chain operations. The team prefers to avoid adjusting the surface area of the system.

I-02: Unused imports, redundant cast, and stale NatSpec across bridge adapters

Files:

aave-delivery-infrastructure/src/contracts/adapters/bob/BobAdapter.sol
aave-delivery-infrastructure/src/contracts/adapters/ink/InkAdapter.sol
aave-delivery-infrastructure/src/contracts/adapters/megaEth/MegaEthAdapter.sol
aave-delivery-infrastructure/src/contracts/adapters/soneium/soneiumAdapter.sol
aave-delivery-infrastructure/src/contracts/adapters/xLayer/xLayerAdapter.sol
aave-delivery-infrastructure/src/contracts/adapters/linea/LineaAdapter.sol
aave-delivery-infrastructure/src/contracts/adapters/zkSync/ZkSyncAdapter.sol

Description:

- **Unused `BaseAdapter` / `Errors` imports.** The struct-based Optimism wrappers (`BobAdapter`, `InkAdapter`, `MegaEthAdapter`, `soneiumAdapter`, `xLayerAdapter`) each import `{BaseAdapter, IBaseAdapter}` from `'../BaseAdapter.sol'` and import `{Errors} ...`, but only reference `IBaseAdapter` (for the `TrustedRemotesConfig` struct field), `ChainIds`, `OpAdapter`, and `IOpAdapter`. The `BaseAdapter` symbol and `Errors` are never used in these files.
- **Unused `SafeCast` import.** `LineaAdapter` imports `SafeCast` but never uses it – Linea is fee-based and performs no gas-limit cast.
- **Redundant cast.** `ZkSyncAdapter` writes `uint256(tx.gasprice)`, `tx.gasprice` is already `uint256`.
- **Stale NatSpec.** `soneiumAdapter.sol` documents the constructor param as *"ink entry point address"* – a copy-paste from `InkAdapter`. It should read "Soneium".

Recommendations:

Consider removing the unused `BaseAdapter`, `Errors`, and `SafeCast` imports, drop the redundant `uint256(...)` cast in `ZkSyncAdapter`, and correct the Soneium NatSpec.

Customer Response:

Acknowledged; no code change will be made. All of these items were already present in the original a.DI and do not affect the adapters in use, with one exception (a quoted line in `ZkSyncAdapter`, which was copied from the existing forward path). To avoid unnecessarily

enlarging the diff, these pre-existing issues in files that were not otherwise modified are left unchanged.

I-03: Cross-block quote/forward fee mismatch can revert sends or strand funds, since random bridge selection changes each block

Files:

aave-delivery-infrastructure/src/contracts/CrossChainForwarder.sol

aave-delivery-infrastructure/src/contracts/libs/Utils.sol

Description:

When `optimalBandwidth < adapterCount`, both `forwardMessage` and `quoteForwardMessage` pick a random adapter subset via `shuffleArray`, seeded by `getPseudoRandom = keccak256(block.number-1, blockhash(block.number-1), entropy)`.

The seed is block-deterministic with no tx/caller entropy, so the selected subset – and therefore the quoted fee – changes from block to block. A quote and a forward only agree when executed in the **same block**, an off-chain quote taken in an earlier block can select a different subset, and since per-bridge fees differ this can under-fund (some sends fail → `forwardingSuccesses` may drop below `requiredForwardingSuccesses` → revert) or over-fund (surplus stranded in the controller).

The protocol's own `AdiAdapter` is immune: it quotes and forwards atomically in the same block. Exposure is limited to external integrators quoting in one block and forwarding in a later one.

Recommendations:

Document that faithful quoting requires same-block quote + forward since the quoted fees can change between blocks.

Customer Response:

Acknowledged, won't fix. `ICrossChainForwarder` quote NatSpec already states quotes are point-in-time, shuffles and fees can change block-to-block, and on-chain callers must use the same `quoteBandwidth` as the forward. Stable Vault's `AdiAdapter` quotes and forwards in one call, so this only affects external integrators, who are covered by that documentation.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.